Crime Records of India - Development Context and Progress Report

Last updated: August 15, 2026

Project overview

Crime Records of India is a dynamic crime-data analytics and governance backend built with Node.js, Express.js, MySQL, Redis, Docker, and JWT authentication.

The system currently supports:

- User and administrator authentication
- Dynamic dataset discovery and schema APIs
- Paginated dataset browsing, filtering, sorting, and search
- Analytics and activity tracking
- Administrator dataset governance
- Transactional audit logging
- Redis metadata and analytics caching
- Role-based access control

Technology stack

- Backend: Node.js and Express.js
- Database: MySQL with mysql2/promise
- Cache: Redis 7
- Authentication: JWT access and refresh tokens, bcrypt password hashes
- Infrastructure: Docker Compose for Redis; backend and MySQL run locally
- Testing: Node.js built-in test runner

Authentication system

Implemented:

- User registration
- User login using username and password
- Admin login using username and password
- JWT access and refresh token generation
- Refresh-token persistence and revocation
- Protected profile API
- Role-based administrator middleware
- User and administrator last_login updates after successful authentication
- Rejection of inactive or suspended administrator accounts

Administrator roles accepted by governance routes:

- superadmin
- admin
- moderator

Dataset engine

Implemented APIs:

- GET /api/tables
- GET /api/tables/:table/schema
- GET /api/data/:table
- GET /api/data/:table/filter-options/:column
- POST /api/data/:table/search

Current behavior:

- Dynamic discovery of dataset tables
- Exact schema discovery through information_schema
- Pagination with a maximum page size of 100
- Exact-match filters and null-value filtering
- Validated dynamic sorting
- View logging in views_log
- Search logging in search_logs
- Redis caching for table and schema metadata
- System tables blocked from every dataset route, including direct URL access
- Dynamic column names validated before being used as SQL identifiers

Admin governance system

Implemented APIs:

- PUT /api/admin/table/:table/update
- DELETE /api/admin/table/:table/delete
- GET /api/admin/audit-logs
- GET /api/admin/audit-logs/:id

Mutation behavior:

- Only dataset tables can be modified; authentication, governance, analytics, metadata, notification, and upload tables are protected.
- Condition and update fields must match real columns in the selected dataset.
- SQL identifiers are escaped after schema validation.
- Null conditions use SQL IS NULL semantics.
- Matching rows are locked with SELECT ... FOR UPDATE.
- Dataset mutation and audit insertion run in the same MySQL transaction.
- If audit logging fails, the dataset change is rolled back.
- A condition matching multiple rows returns 409 until the caller sends confirmMultiple: true.
- Update requests use newData; the existing updateData name remains a compatibility alias.

Audit records capture:

- Admin id and username
- Dataset table name
- Row identifier or condition summary
- UPDATE or DELETE action
- Complete old row snapshots
- Complete expected new row snapshots for updates
- Changed or deleted columns
- Admin IP address
- Creation timestamp

Audit-log list filters:

- Dataset table
- Action type
- Admin id
- Start and end date
- Pagination

The live admin_modification_logs table already contains the required JSON and tracking columns, so no database migration was required for this implementation.

Analytics system

Implemented:

- Total user count
- Total administrator count
- Total dataset table count
- Total view count
- Total search count
- Most viewed dataset
- User logins during the previous seven days

API:

- GET /api/analytics/overview

Cache TTL: 300 seconds.

Redis cache layer

Current cache keys:

Cache key	Purpose	TTL
all_tables	Dataset table list	1800 seconds
schema_{table}	Dataset schema	3600 seconds
analytics_overview	Analytics overview	300 seconds

Only Redis runs in Docker. The backend and MySQL run locally.

Current database tables

Core and platform tables:

- users
- admins
- refresh_tokens
- user_settings
- notifications
- dataset_metadata
- dataset_columns
- views_log
- search_logs
- admin_modification_logs
- api_request_logs
- upload_history
- response

Dataset tables found in the live database:

- auto_theft
- cases_under_crime_against_women
- ch_not_murder_victim_age_sex
- complaints_against_police
- custodial_death_during_hospitalization_or_treatment
- custodial_death_others
- custodial_death_person_not_remanded
- custodial_death_person_remanded
- human_rights_violation_by_police
- murder_victim_age_sex
- period_of_trials_by_courts
- police_housing
- property_stolen_and_recovered
- serious_fraud
- specific_purpose_of_kidnapping_and_abduction
- trial_of_violent_crimes_by_courts
- victims_of_rape

Security behavior

Implemented:

- Helmet headers
- CORS middleware
- JWT authentication
- Admin role authorization
- Dataset/system-table separation
- Dynamic column allow-listing
- Parameterized values in dynamic SQL
- Transactional audit logging
- Explicit bulk-mutation confirmation
- Generic server errors without returning raw SQL details

The in-memory rate-limiter middleware exists but is not yet mounted globally.

Verification completed

- npm test passes all dataset-table safety tests.
- JavaScript syntax checks pass for changed source files.
- Live MySQL schema discovery succeeds.
- A live update plus audit insert was executed inside a transaction and rolled back successfully, leaving no verification data behind.
- The live check covered the Auto_Theft_Coordinated/Traced column to verify safe handling of unusual identifier characters.
- System-table guards were verified for lowercase and uppercase direct access.
- Audit-log list and detail queries were verified against existing audit data.

Local development commands

Backend:

```
cd backend
```

```
npm run dev
```

Tests:

```
cd backend
npm test
```

Redis with Docker Compose:

```
cd backend
docker compose up -d
```

Alternative Redis command:

```
docker run -d --name crime-redis -p 6379:6379 redis:7
```

Project status

- User authentication: working
- Admin authentication and status enforcement: working
- Dynamic dataset APIs: working
- Dataset route isolation: working
- Admin update/delete governance APIs: working
- Transactional audit logging: working
- Audit-log read APIs: working
- Analytics: working
- Redis cache integration: working when Redis is available
- Admin frontend: pending

Remaining work

- Build and integrate the admin frontend
- Mount or replace the current rate limiter for production use
- Add request-schema validation middleware for non-dataset endpoints
- Add user view-history APIs
- Add OpenAPI/Swagger generation
- Add deployment configuration and production Dockerization
- Add broader HTTP integration tests with isolated test database fixtures

Frontend handoff notes

- Use the dataset schema endpoint to build editable table columns dynamically.
- Submit updates with conditions and newData.
- On a 409 response, show the matched-row count and require administrator confirmation before resubmitting with confirmMultiple: true.
- Use the audit list endpoint for the governance activity table.
- Use the audit detail endpoint to display old values, new values, changed columns, actor, IP address, and timestamp.
- The complete payload and response contracts are documented in Crime_Records_API_Documentation (1).md.